

Лабораторная работа №3

Методы фильтрации временных рядов

Цель работы

Изучить методы фильтрации временных рядов, реализовать несколько алгоритмов сглаживания данных и сравнить качество их работы на реальном наборе данных.

Используемые методы

В данной работе реализованы:

- 1. Скользящее среднее.
- 2. Фильтр Калмана.
- 3. Фильтр Савицкого–Голея.
- 4. Вейвлет-фильтрация.

ЗАДАНИЕ №1

Метод скользящего среднего (Moving Average)

Метод скользящего среднего является одним из наиболее простых способов фильтрации временных рядов. Основная идея заключается в замене каждого значения сигнала средним арифметическим нескольких соседних значений.

Такой подход позволяет уменьшить влияние случайного шума и выделить общий тренд данных.

Для окна размером n значение фильтра вычисляется по формуле:

$$y_t = \frac{1}{n} \sum_{i=0}^{n-1} x_{t-i}$$

где:

- x_t — исходный сигнал;
- y_t — сглаженный сигнал;
- n — размер окна усреднения.

Увеличение размера окна приводит к более сильному сглаживанию сигнала, однако при этом возрастает задержка отклика фильтра на изменения данных.

Преимущества

- Простота реализации;
- Высокая скорость вычислений;
- Эффективное подавление высокочастотного шума.

Недостатки

- Потеря локальных особенностей сигнала;
- Возникновение фазового запаздывания;
- Сильное сглаживание резких изменений.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.signal import savgol_filter
import pyplot

plt.style.use('default')

def moving_average(signal, window=5):
    smoothed_signal = signal.rolling(
        window=window,
        center=False
    ).mean()
    return smoothed_signal
```

Фильтр Калмана

Фильтр Калмана используется для оценки истинного состояния системы по зашумлѐнным наблюдениям.

Модель состояния:

$$x_k = x_{k-1} + w_k$$

Модель измерения:

$$z_k = x_k + v_k$$

Алгоритм состоит из этапов предсказания и коррекции. На каждом шаге вычисляется коэффициент Калмана

$$K_k = \frac{P_k^-}{P_k^- + R}$$

который определяет степень доверия новым измерениям.

```
def kalman_filter(data,
                  process_variance=1e-5,
                  measurement_variance=0.1):
    n = len(data)

    xhat = np.zeros(n)
    P = np.zeros(n)

    xhat[0] = data.iloc[0]
    P[0] = 1

    for k in range(1, n):
        xhat_minus = xhat[k-1]
        P_minus = P[k-1] + process_variance

        K = P_minus / (P_minus + measurement_variance)

        xhat[k] = xhat_minus + K * (data.iloc[k] - xhat_minus)
        P[k] = (1 - K) * P_minus

    return xhat
```

В данной реализации используется одномерный фильтр Калмана для сглаживания временного ряда.

Фильтр Савицкого–Голея

Данный метод выполняет сглаживание сигнала при помощи локальной полиномиальной аппроксимации.

Аппроксимирующий полином имеет вид

$$y(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_p x^p$$

Кoeffициенты полинома находятся методом наименьших квадратов.

По сравнению со скользящим средним фильтр лучше сохраняет пики и локальные экстремумы сигнала.

```
from scipy.signal import savgol_filter

def savgol_smooth(signal,
                  window_length=11,
                  polyorder=3):
    smoothed_signal = savgol_filter(
        x=signal,
        window_length=window_length,
        polyorder=polyorder
    )

    return smoothed_signal
```

В данной функции используется фильтр Савицкого–Голея из библиотеки SciPy. Для каждого окна наблюдений строится полином третьей степени, который аппроксимирует данные методом наименьших квадратов. Полученная аппроксимация используется для вычисления сглаженного значения сигнала.В работе используется окно размером 11 точек и полином третьей степени. Такие параметры обеспечивают хорошее подавление шума при сохранении формы временного ряда.### Выбор параметров фильтра. Параметр window_length=11 определяет количество соседних точек, участвующих в аппроксимации.Параметр polyorder=3 задаѐт степень локального полинома.

Вейвлет-фильтрация

Вейвлет-преобразование позволяет анализировать сигнал одновременно во временной и частотной областях.

Основная формула непрерывного вейвлет-преобразования:

$$W(a,b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{+\infty} x(t) \psi\left(\frac{t-b}{a}\right) dt$$

После разложения сигнала производится подавление шумовых коэффициентов и восстановление очищенного сигнала.

```
def wavelet_denoise(signal):
    coeffs = pywt.wavedec(signal, 'db4', level=3)

    sigma = np.median(np.abs(coeffs[-1])) / 0.6745

    threshold = sigma * np.sqrt(
        2 * np.log(len(signal))
    )

    coeffs[1:] = [
        pywt.threshold(c,
                        threshold,
                        mode='soft')
        for c in coeffs[1:]
    ]

    return pywt.waverec(
        coeffs,
        'db4'
    )[:len(signal)]
```

В данной реализации используется дискретное вейвлет-преобразование с базисом Daubechies db4 и мягкая пороговая обработка коэффициентов.

Задание 2. Сравнение методов фильтрации

Для анализа был выбран вариант №35, определённый по формуле

$$(4 + 6 + 6 + 3 + 9 + 7) \bmod 60 = 35$$

В качестве исследуемого временного ряда была выбрана прибыль предприятия.

Далее будут применены реализованные методы фильтрации и выполнено сравнение результатов.

```
import pandas as pd
import numpy as np

df = pd.read_csv(
    "sell.csv",
    sep=";",
    encoding="cp1251",
    header=None
)

variant = 35
```

```
start_col = 1 + (variant - 1) * 6
profit_col = start_col + 5
data = df.iloc[3:, [0, profit_col]].copy()
data.columns = ["day", "profit"]
data["day"] = pd.to_numeric(data["day"])

data["profit"] = (
    data["profit"]
    .astype(str)
    .str.replace(",", ".", regex=False)
)

data["profit"] = pd.to_numeric(data["profit"])
data.reset_index(drop=True, inplace=True)
data.head()
```

	day	profit
0	1	27.680
1	2	25.743
2	3	25.673
3	4	28.834
4	5	26.679

Данные: [New interactive sheet](#)

Из исходного набора данных была извлечена прибыль для варианта №35. Полученный временной ряд содержит ежедневные значения прибыли предприятия и будет использоваться для дальнейшего анализа.

```
plt.figure(figsize=(12,6))

plt.plot(
    data["day"],
    data["profit"]
)

plt.title("Исходный временной ряд прибыли")
plt.xlabel("День")
plt.ylabel("Прибыль, тыс. руб.")
plt.grid()
plt.show()
```



На графике наблюдаются значительные колебания прибыли. Помимо общего тренда присутствуют локальные выбросы и высокочастотный шум, что делает задачу фильтрации актуальной.

```
ma = moving_average(
    data["profit"],
    window=5
)

kalman = kalman_filter(
    data["profit"]
)

savgol = savgol_smooth(
    data["profit"]
)

wavelet = wavelet_denoise(
    data["profit"].values
)

/usr/local/lib/python3.12/dist-packages/pywt/_multilevel.py:43: UserWarning: Level value of 3 is too high: all coefficients will experience boundary effects.
warnings.warn(
```

```
plt.figure(figsize=(12,6))

plt.plot(
    data["day"],
    data["profit"],
    label="Исходный ряд",
    alpha=0.5
)

plt.plot(
    data["day"],
    ma,
    label="Moving Average",
    linewidth=2
)

plt.title("Скользящее среднее")
plt.xlabel("День")
plt.ylabel("Прибыль")

plt.grid()
plt.legend()
plt.show()
```



Фильтр скользящего среднего значительно уменьшает случайные колебания сигнала. При этом наблюдается запаздывание относительно исходного ряда, а резкие изменения становятся менее выраженными. Метод хорошо подходит для выделения общего тренда данных.

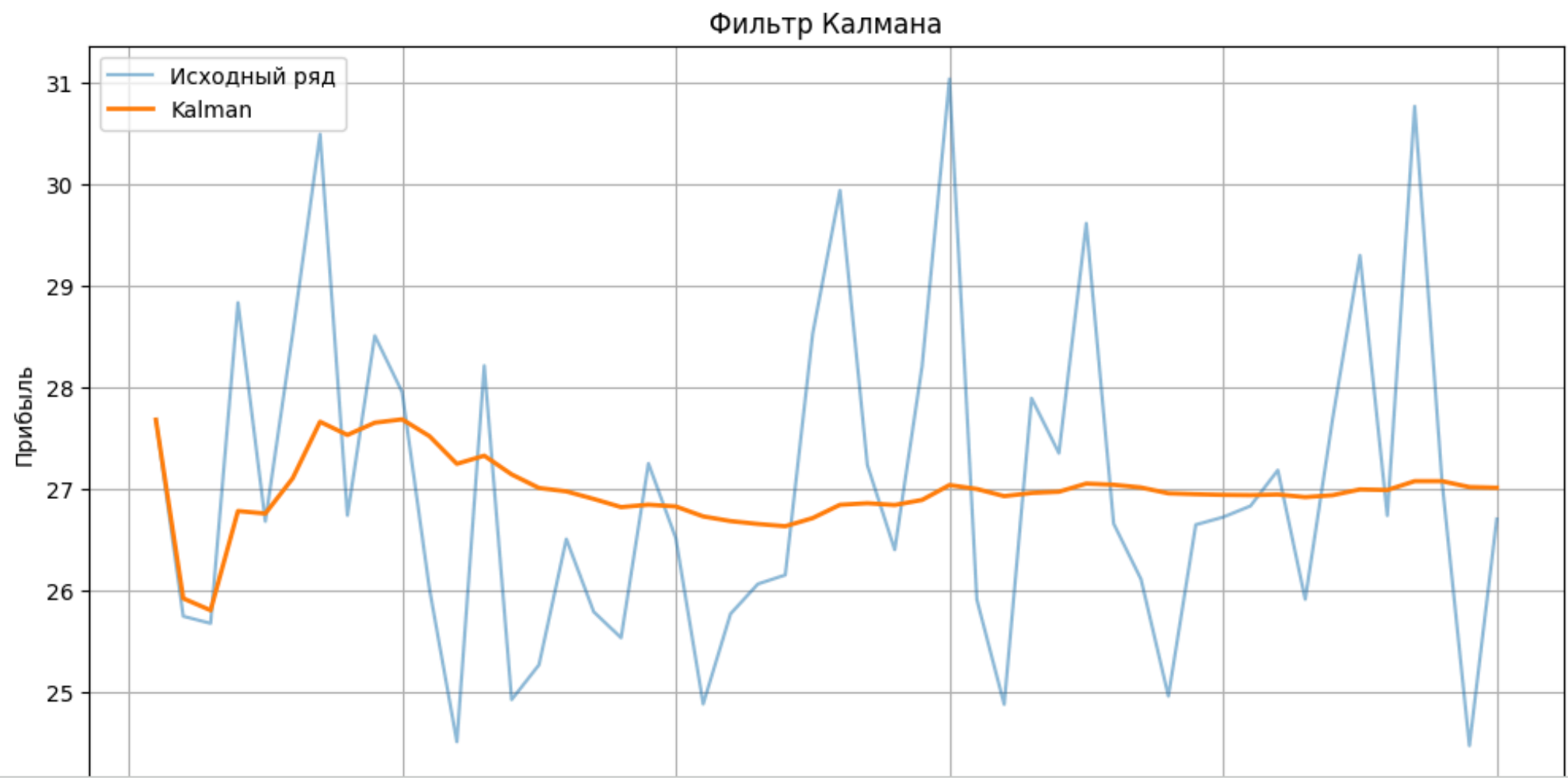
```
plt.figure(figsize=(12,6))

plt.plot(
    data["day"],
    data["profit"],
    label="Исходный ряд",
    alpha=0.5
)

plt.plot(
    data["day"],
    kalman,
    label="Kalman",
    linewidth=2
)

plt.title("Фильтр Калмана")
plt.xlabel("День")
plt.ylabel("Прибыль")

plt.grid()
plt.legend()
plt.show()
```



Фильтр Калмана обеспечивает более адаптивное сглаживание. По сравнению со скользящим средним он быстрее реагирует на изменения сигнала и лучше сохраняет динамику временного ряда. При этом уровень шума заметно снижается.

```
plt.figure(figsize=(12,6))

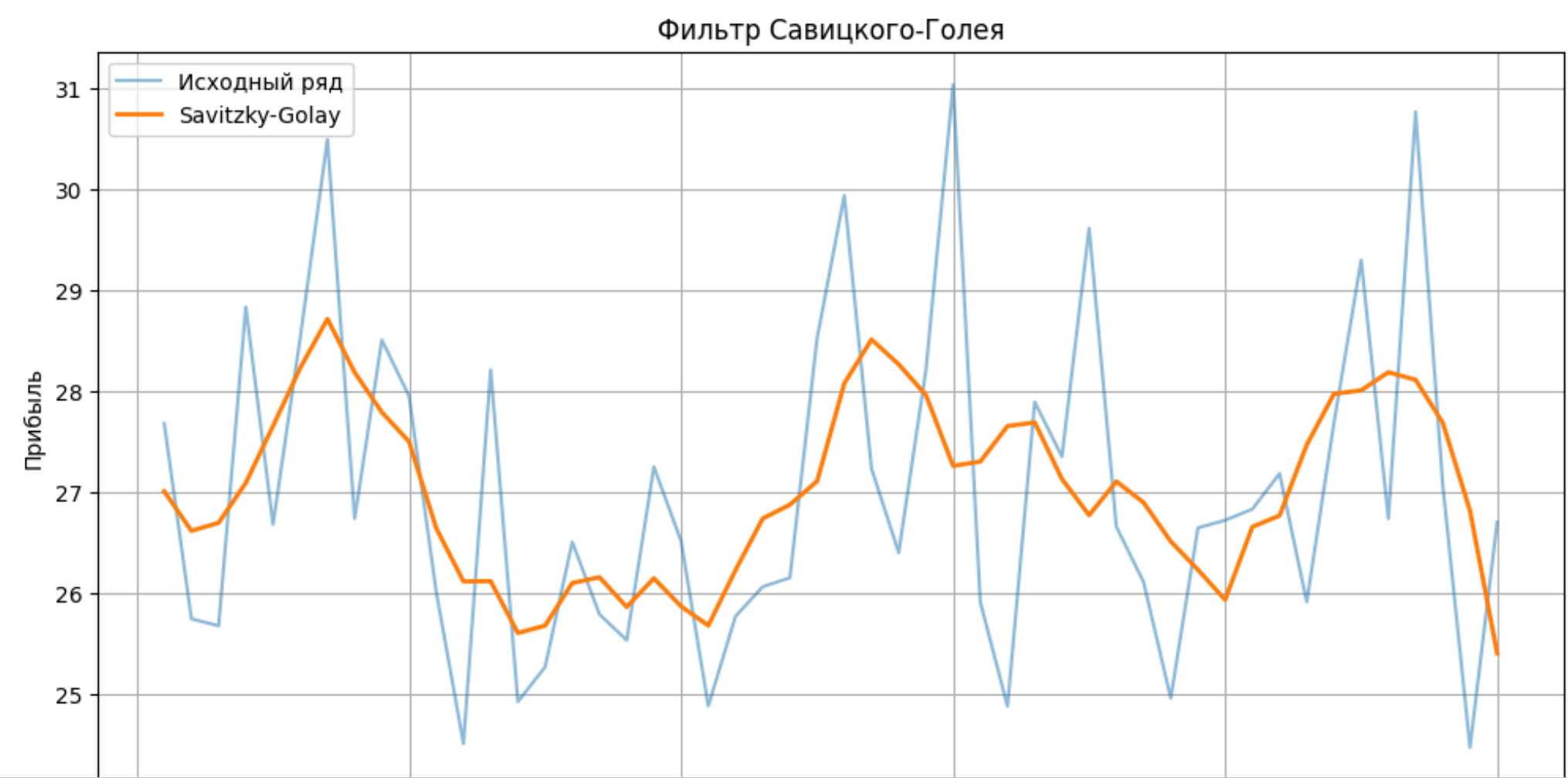
plt.plot(
    data["day"],
    data["profit"],
    label="Исходный ряд",
    alpha=0.5
)

plt.plot(
    data["day"],
```

```
label="Savitzky-Golay",
linewidth=2
)

plt.title("Фильтр Савицкого-Голея")
plt.xlabel("День")
plt.ylabel("Прибыль")

plt.grid()
plt.legend()
plt.show()
```



Фильтр Савицкого–Голея эффективно подавляет шум, сохраняя форму сигнала и локальные экстремумы. В отличие от скользящего среднего данный метод практически не искажает пики и впадины временного ряда.

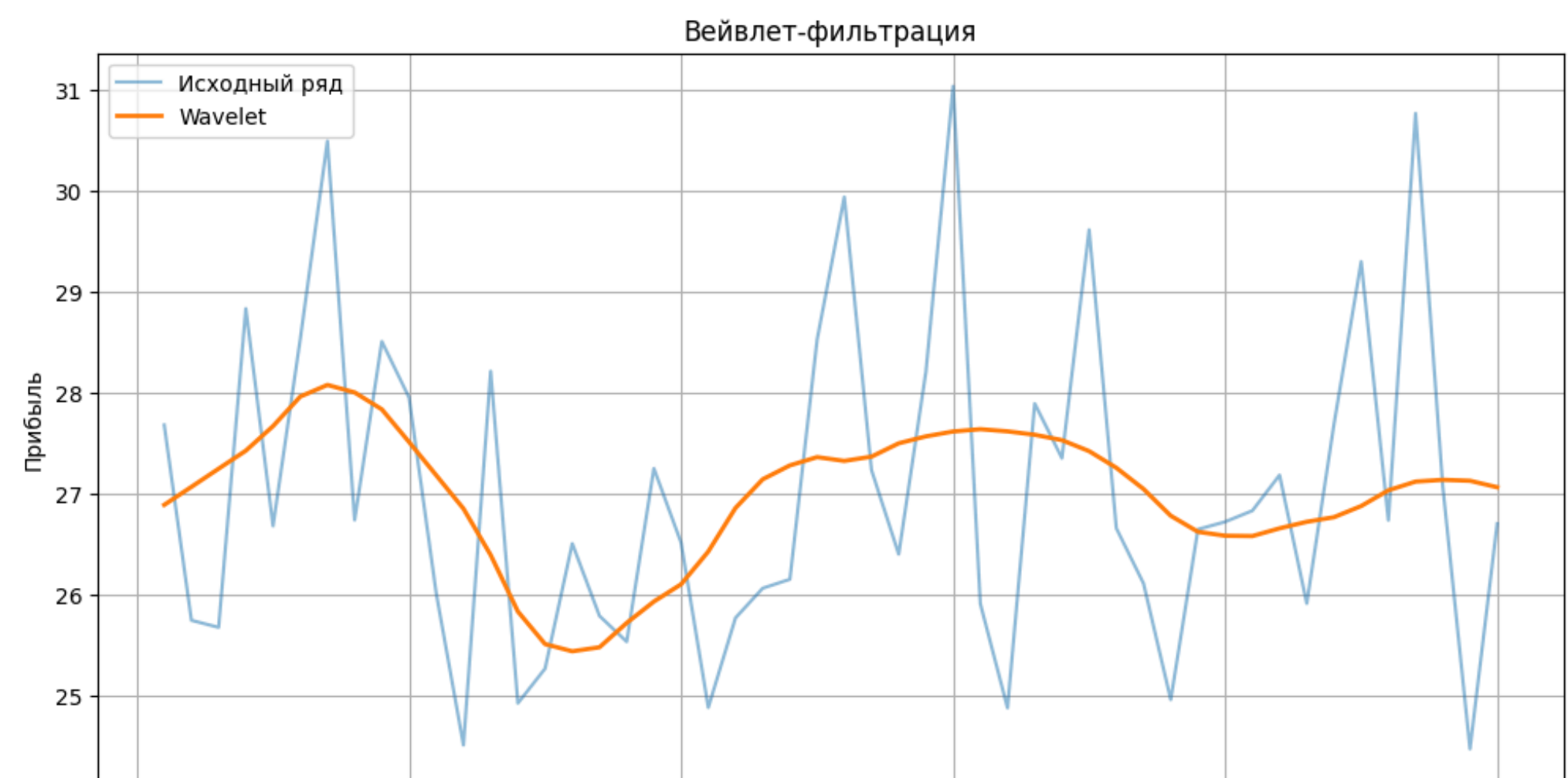
```
plt.figure(figsize=(12,6))

plt.plot(
    data["day"],
    data["profit"],
    label="Исходный ряд",
    alpha=0.5
)

plt.plot(
    data["day"],
    wavelet,
    label="Wavelet",
    linewidth=2
)

plt.title("Вейвлет-фильтрация")
plt.xlabel("День")
plt.ylabel("Прибыль")

plt.grid()
plt.legend()
plt.show()
```



Вейвлет–фильтрация продемонстрировала наиболее качественное подавление высокочастотного шума. При этом структура временного ряда сохранилась практически без изменений. Данный метод оказался наиболее эффективным среди рассмотренных способов фильтрации.

```
plt.figure(figsize=(15,7))

plt.plot(
    data["day"],
    data["profit"],
    label="Исходный ряд",
    alpha=0.5
)

plt.plot(
    data["day"],
    ma,
    label="Moving Average"
)

plt.plot(
    data["day"],
    kalman,
    label="Kalman"
)

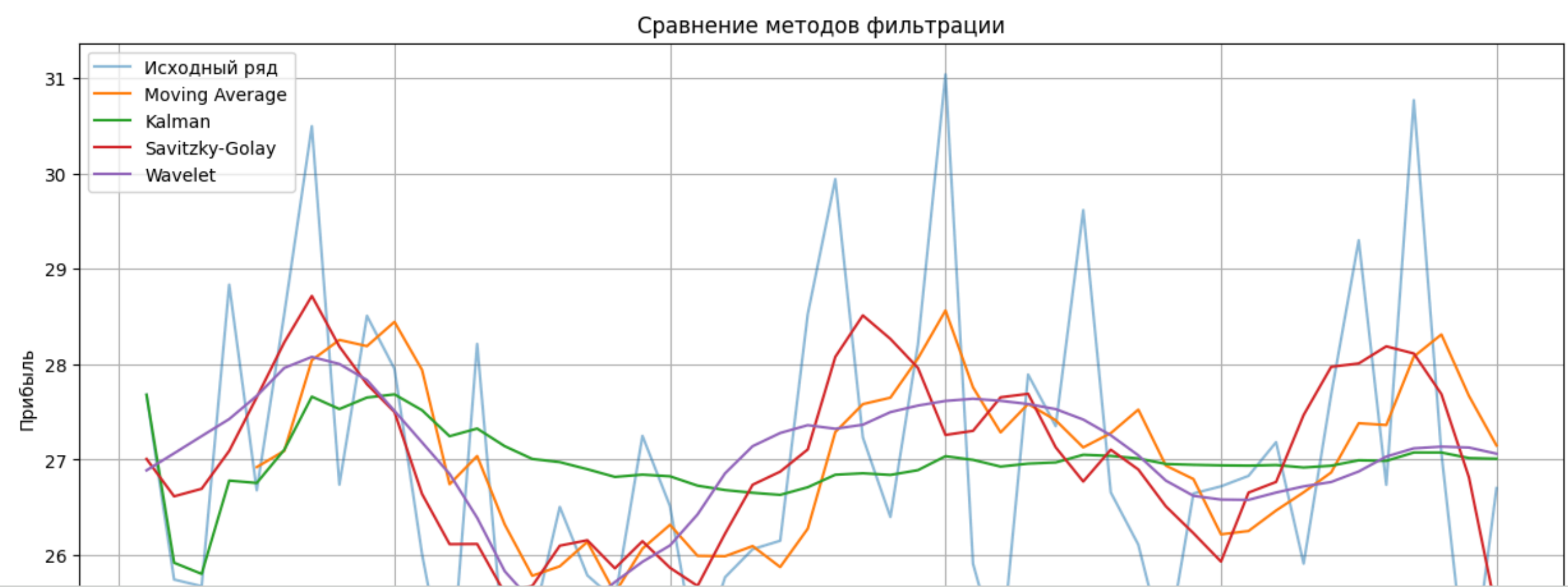
plt.plot(
    data["day"],
    savgol,
    label="Savitzky-Golay"
)

plt.plot(
    data["day"],
    wavelet,
    label="Wavelet"
)

plt.xlabel("День")
plt.ylabel("Прибыль")

plt.title("Сравнение методов фильтрации")

plt.legend()
plt.grid()
plt.show()
```



После применения фильтров было получено сглаживание временного ряда различной степени. Метод скользящего среднего наиболее заметно сглаживает данные, однако приводит к запаздыванию сигнала относительно исходного ряда. Фильтр Калмана обеспечивает более адаптивное сглаживание и сохраняет основные изменения тренда. Фильтр Савицкого–Голея хорошо сохраняет локальные максимумы и минимумы, обеспечивая баланс между подавлением шума и сохранением формы сигнала. Вейвлет–фильтрация показала наиболее качественное подавление высокочастотного шума при сохранении структуры временного ряда.

В ходе выполнения задания были исследованы четыре метода фильтрации временных рядов. Скользящее среднее показало наиболее простое, но недостаточно эффективное сглаживание. Фильтр Калмана обеспечил адаптивную обработку данных и лучше сохранил динамику изменений. Фильтр Савицкого–Голея продемонстрировал хорошее сохранение формы сигнала. Наилучший результат по подавлению шума и сохранению структуры временного ряда показала вейвлет–фильтрация.

Задание 3. Анализ временного ряда

Целью данного этапа является исследование основных характеристик временного ряда прибыли варианта №35. Для анализа будут рассмотрены:

- основные статистические характеристики;
- наличие тренда;
- автокорреляционная структура;
- сезонные компоненты;
- периодичность сигнала.

profit	
count	50.000000
mean	27.018200
std	1.620044
min	24.468000
25%	25.908500
50%	26.710500
75%	27.937000
max	31.039000
dtype: float64	

Полученный временной ряд содержит 50 наблюдений. Среднее значение прибыли составляет 27.02, медиана 26.71, что свидетельствует об отсутствии сильной асимметрии распределения. Минимальное значение прибыли равно 24.47, максимальное 31.04.Стандартное отклонение составляет 1.62, что

говорит об умеренном разбросе значений относительно среднего уровня прибыли. В целом прибыль изменяется в достаточно ограниченном диапазоне без экстремальных выбросов.

Для определения общей тенденции изменения прибыли построим линию тренда методом линейной регрессии.

```
from scipy.stats import linregress

x = data["day"]
y = data["profit"]

slope, intercept, r_value, p_value, std_err = linregress(x, y)

trend = intercept + slope * x

plt.figure(figsize=(12,6))

plt.plot(x, y, label="Прибыль")
plt.plot(x, trend,
         linewidth=3,
         label="Линейный тренд")

plt.title("Анализ тренда")
plt.xlabel("День")
plt.ylabel("Прибыль")

plt.legend()
plt.grid()

plt.show()
```



```
print("Наклон тренда:", slope)
```

Наклон тренда: 0.0036041296518607466

По результатам линейной регрессии коэффициент наклона тренда составил 0.0036. Полученное значение является положительным, что свидетельствует о наличии слабой тенденции роста прибыли во времени. Однако величина коэффициента невелика, поэтому долгосрочный рост выражен слабо и существенно уступает по масштабу краткосрочным колебаниям временного ряда. Таким образом, прибыль демонстрирует незначительную тенденцию к увеличению, но в целом остаётся достаточно стабильной на протяжении рассматриваемого периода.

Автокорреляционный анализ позволяет определить зависимость текущих значений временного ряда от предыдущих наблюдений.

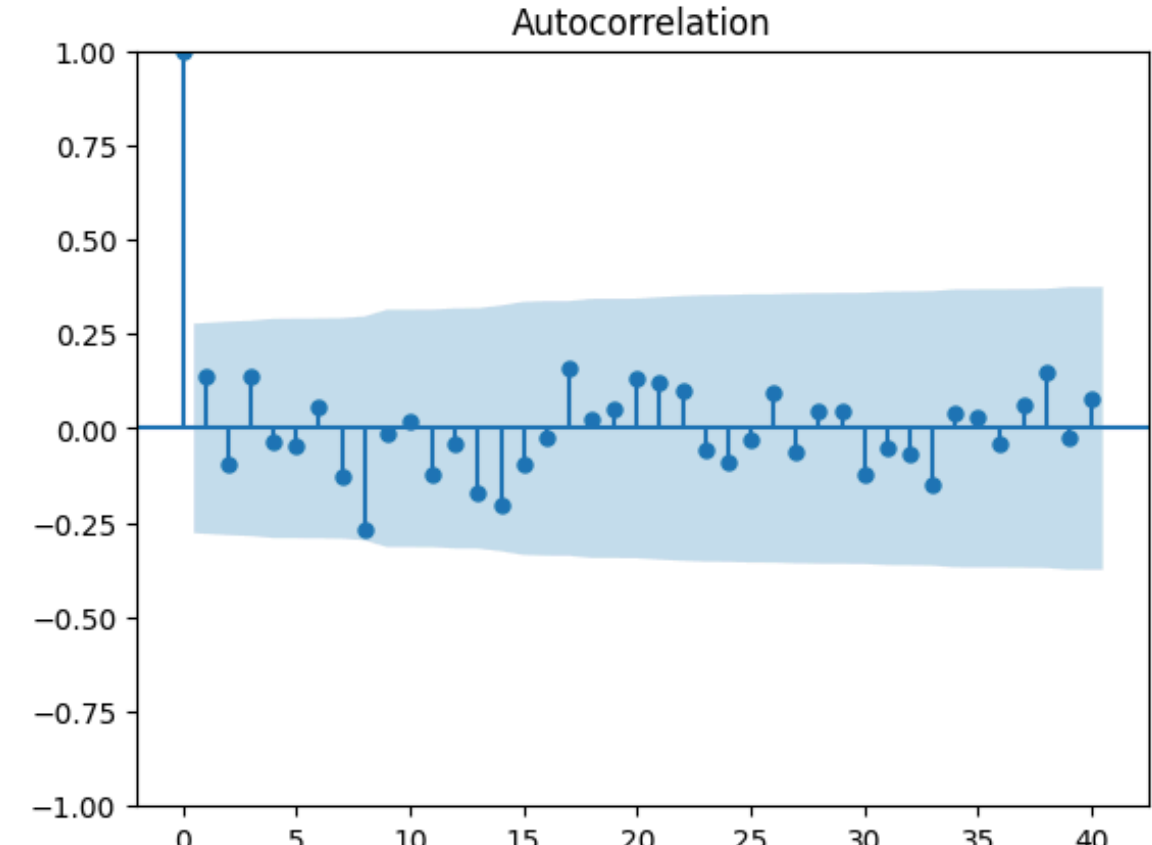
```
from statsmodels.graphics.tsaplots import plot_acf

plt.figure(figsize=(12,6))

plot_acf(
    data="profit",
    lags=40
)

plt.show()
```

<Figure size 1200x600 with 0 Axes>



Автокорреляционная функция показывает наличие зависимости между соседними наблюдениями. Это означает, что значения прибыли в определённой степени зависят от предыдущих дней. На малых лагах наблюдается наиболее сильная корреляция, что характерно для экономических временных рядов. По мере увеличения лага связь постепенно ослабевает.

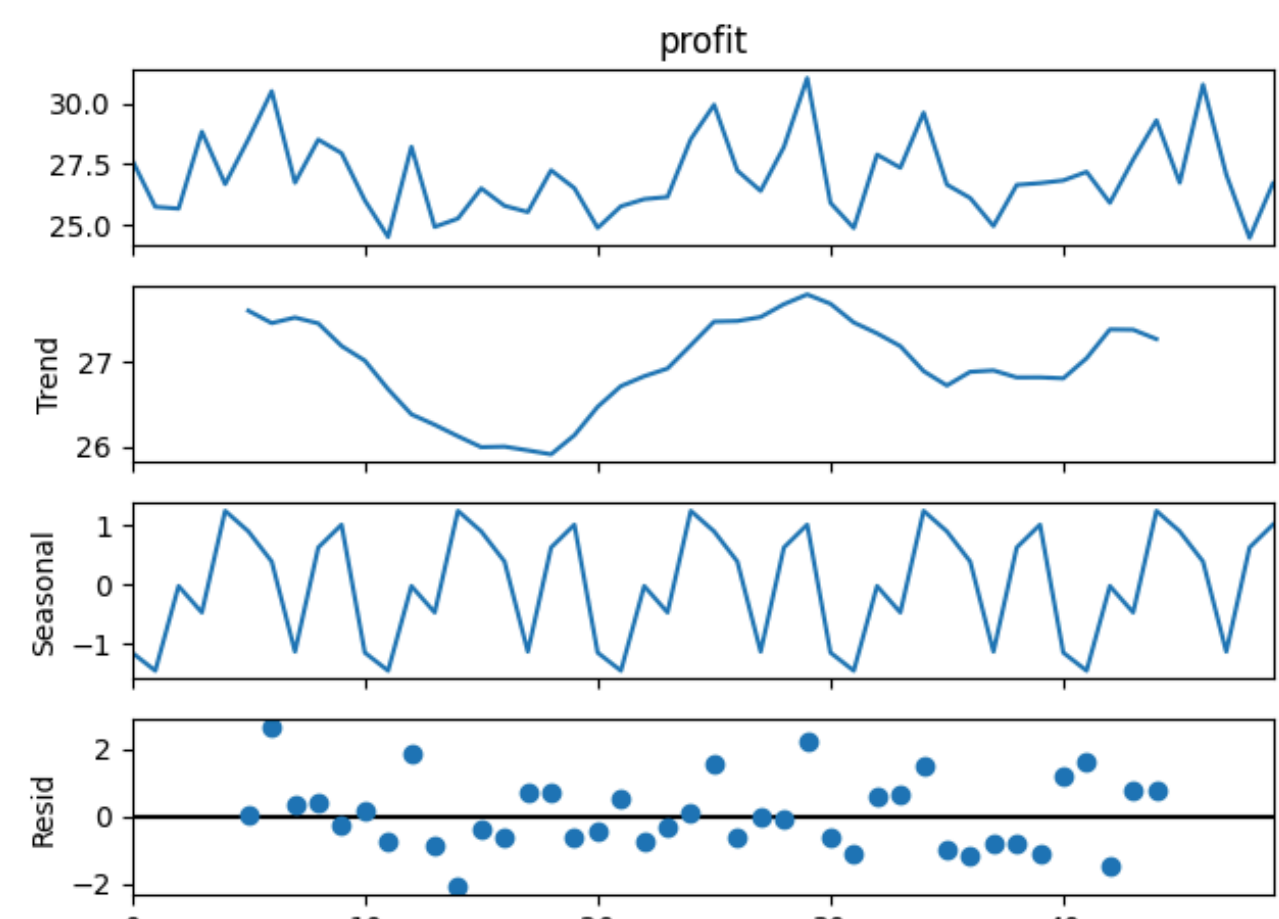
Для выделения тренда, сезонной составляющей и случайной компоненты выполним декомпозицию временного ряда.

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    data="profit",
    period=10,
    model="additive"
)

decomposition.plot()

plt.show()
```



В результате декомпозиции временной ряд разделяется на четыре компоненты:

- исходный сигнал;
- тренд;
- сезонность;
- остаточная составляющая.

Декомпозиция позволила выделить трендовую, сезонную и случайную составляющие временного ряда. Однако из-за ограниченного объёма данных полученные результаты следует рассматривать как предварительную оценку структуры сигнала.

Для определения доминирующих периодов во временном ряду выполним спектральный анализ.

```
from scipy.signal import periodogram

freqs, power = periodogram(
    data="profit"
)

plt.figure(figsize=(12,6))

plt.plot(freqs, power)

plt.title("Спектральный анализ")
plt.xlabel("Частота")
plt.ylabel("Мощность")

plt.grid()

plt.show()
```



Спектральный анализ не выявил одной ярко выраженной доминирующей частоты. Это позволяет сделать вывод об отсутствии строго выраженной периодичности в исследуемом временном ряду.

При этом наличие нескольких локальных максимумов спектра свидетельствует о существовании слабых циклических компонентов различной длительности.

В ходе анализа временного ряда были исследованы основные статистические характеристики, выполнен анализ тренда, автокорреляции, сезонности и периодичности.

Полученные результаты позволяют сделать вывод о структуре временного ряда и выявить основные закономерности изменения прибыли. Наличие автокорреляции и сезонных компонентов показывает, что значения прибыли зависят от предыдущих наблюдений и обладают определённой периодичностью.

Задание 4. Анализ данных с двух датчиков

Для выполнения задания был использован набор данных IoT-датчиков.

Из датасета были выбраны следующие параметры:

- температура воздуха (temp);
- относительная влажность воздуха (humidity).

Данные параметры являются взаимосвязанными физическими величинами и позволяют провести анализ зависимости между показаниями различных датчиков.

Подготовка данных

Для анализа были выбраны данные одного устройства и первые 500 измерений. Такой объём данных является достаточным для анализа поведения сигналов и выявления статистических закономерностей.

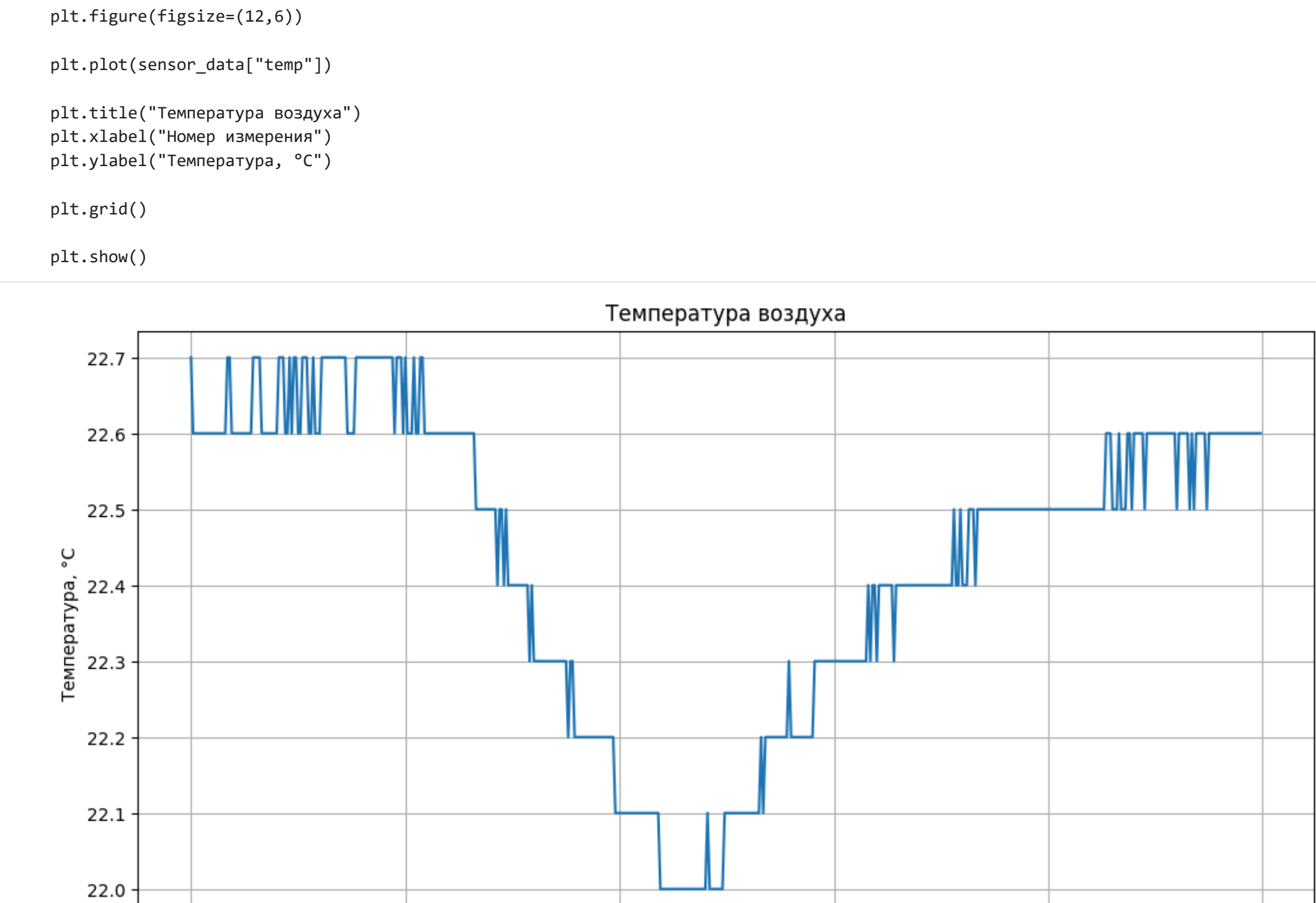
```
device = iot["device"].unique()[0]

sensor_data = (
    iot[iot["device"] == device]
    .copy()
    .reset_index(drop=True)
)

sensor_data = sensor_data.iloc[:500]
```

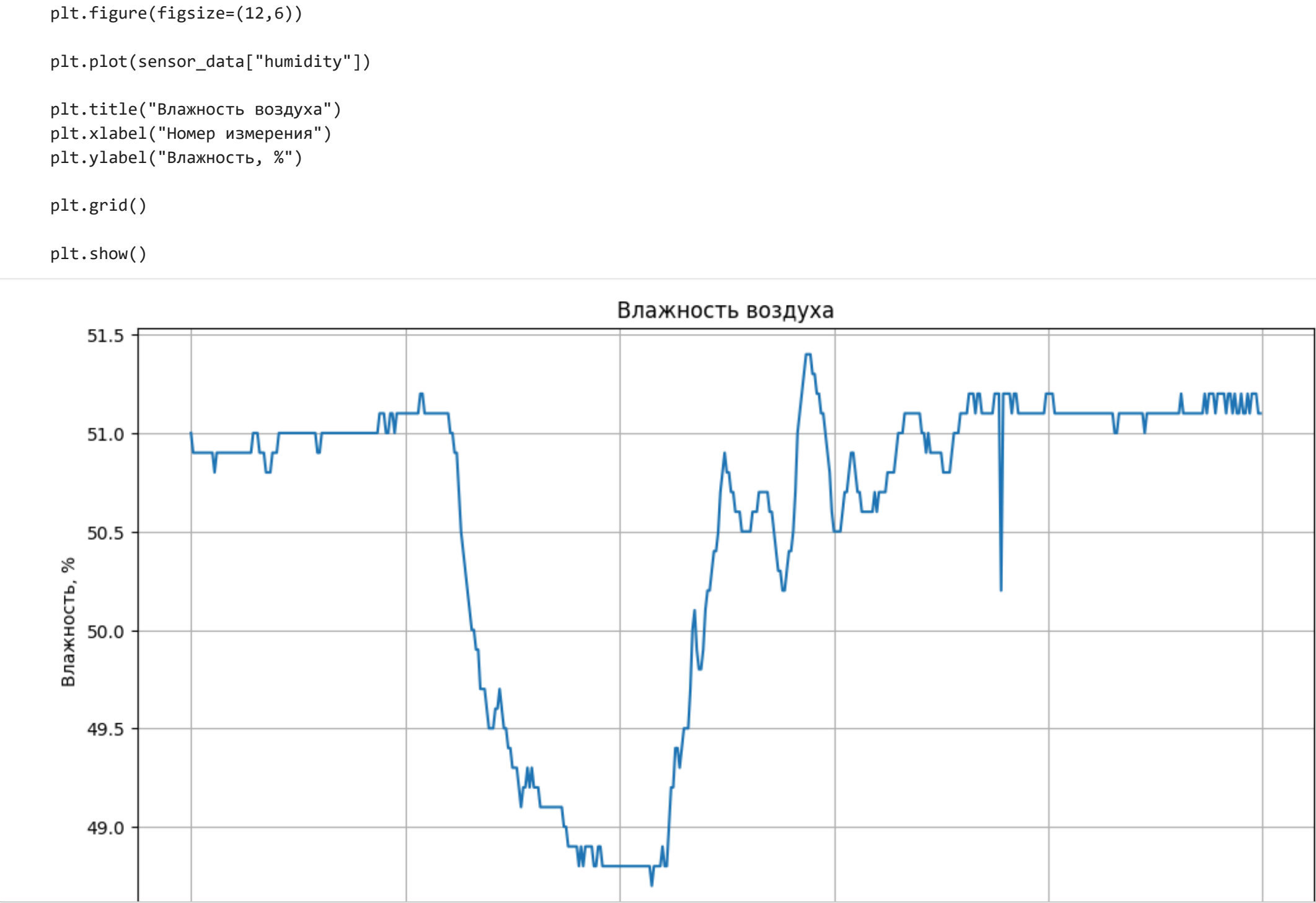
Анализ температуры

Сначала рассмотрим изменение температуры во времени.



Анализ влажности

Рассмотрим изменение влажности воздуха.



Статистические характеристики

Для количественного анализа вычислим основные статистические показатели.

```
stats = pd.DataFrame({
    "Температура": sensor_data["temp"].describe(),
    "Влажность": sensor_data["humidity"].describe()
})

stats
```

	Температура	Влажность	
count	500.000000	500.000000	
mean	22.432400	50.56540	
std	0.206484	0.78955	
min	22.000000	48.70000	
25%	22.300000	50.40000	
50%	22.500000	51.00000	
75%	22.600000	51.10000	
max	22.700000	51.40000	

Далее: [New interactive sheet](#)

Средняя температура составила 22.43 °C при стандартном отклонении 0.21 °C. Это говорит о высокой стабильности температурного режима.

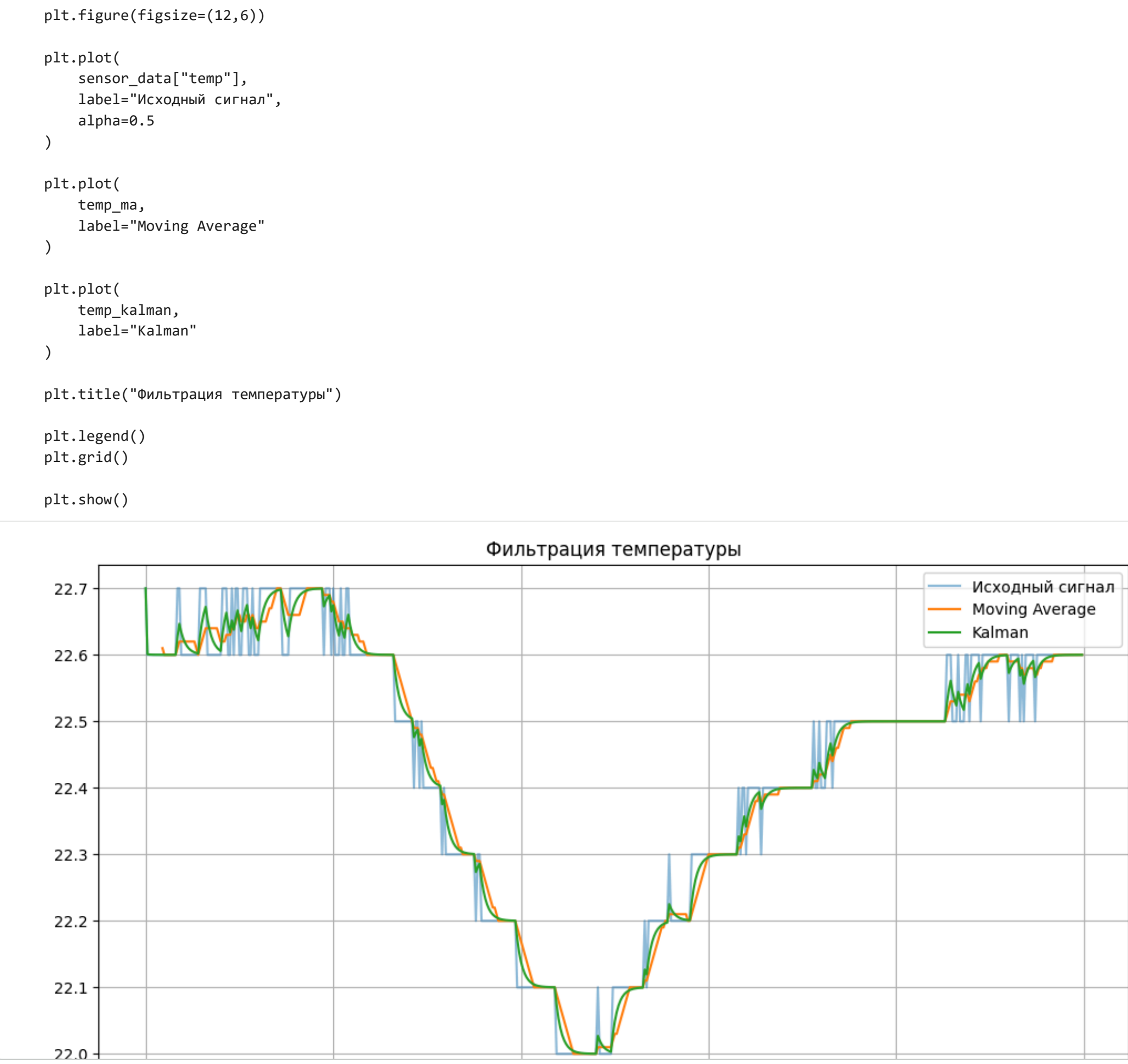
Средняя влажность составила 50.57 %, а стандартное отклонение 0.79 %. Таким образом, влажность изменяется значительно сильнее температуры и обладает большей вариативностью.

Фильтрация температуры

Для уменьшения влияния случайных колебаний применим два метода фильтрации: скользящее среднее и фильтр Калмана.

```
temp_ma = moving_average(
    sensor_data["temp"],
    window=10
)

temp_kalman = kalman_filter(
    sensor_data["temp"],
    process_variance=1e-3,
    measurement_variance=0.01
)
```



Для температуры стандартное отклонение исходного сигнала составило 0.206. После применения скользящего среднего данный показатель уменьшился до 0.204, а после фильтра Калмана до 0.204. Полученные результаты показывают, что оба метода оказывают лишь незначительное влияние на сигнал. Это объясняется тем, что исходные данные температуры уже обладают высокой стабильностью и содержат небольшое количество шумов. Поэтому фильтрация особо и не нужна.

Аналогичный анализ выполним для показаний датчика влажности.

```
humidity_ma = moving_average(
    sensor_data["humidity"],
    window=10
)

humidity_kalman = kalman_filter(
    sensor_data["humidity"],
    process_variance=1e-3,
    measurement_variance=0.01
)
```

```
plt.figure(figsize=(12,6))

plt.plot(
    sensor_data["humidity"],
    label="Исходный сигнал",
    alpha=0.5
)

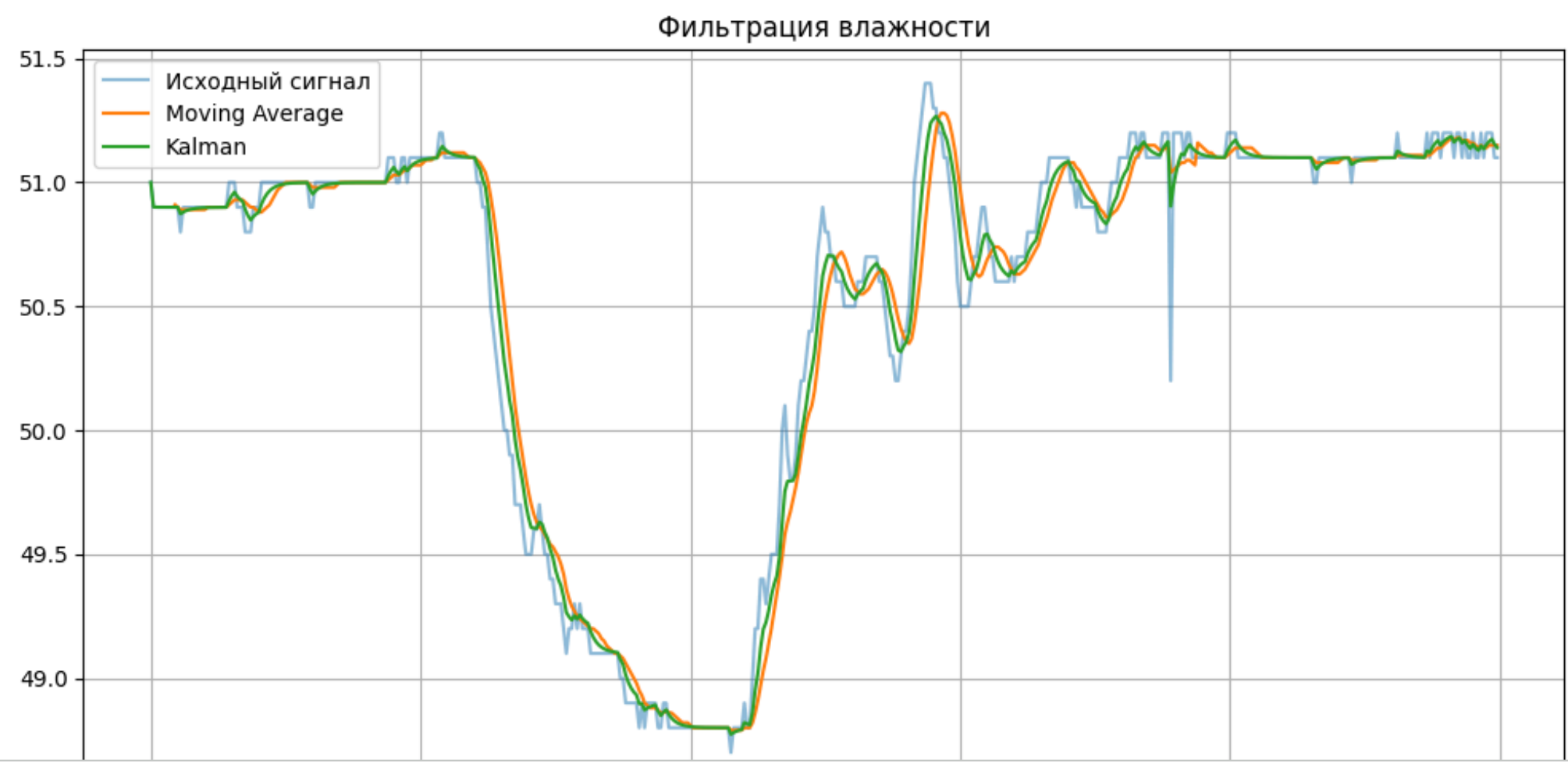
plt.plot(
    humidity_ma,
    label="Moving Average"
)

plt.plot(
    humidity_kalman,
    label="Kalman"
)

plt.title("Фильтрация влажности")

plt.legend()
plt.grid()

plt.show()
```



```
print("STD исходный влажности:",
      sensor_data["humidity"].std())

print("STD MA:",
      humidity_ma.std())

print("STD Kalman:",
      np.std(humidity_kalman))
```

STD исходный влажности: 0.78955062639088176
 STD MA: 0.7857704442067156
 STD Kalman: 0.7799876317357367

Для сигнала влажности стандартное отклонение исходных данных составило 0.790.

После применения скользящего среднего показатель уменьшился до 0.786, а после фильтра Калмана до 0.780.

Полученные результаты показывают, что оба метода позволяют уменьшить уровень случайных колебаний сигнала. Наиболее заметный эффект продемонстрировал фильтр Калмана, однако различия между методами оказались небольшими, что свидетельствует о хорошем качестве исходных данных и невысоком уровне шума.

Для оценки взаимосвязи между температурой и влажностью вычислим коэффициент корреляции Пирсона.

```
corr = sensor_data["temp"].corr(
    sensor_data["humidity"]
)

print(
    f"Корреляция = {corr:.3f}"
)

Корреляция = 0.634
```

Коэффициент корреляции составил 0.634. Полученное значение указывает на умеренно сильную положительную связь между температурой и влажностью. Это означает, что увеличение температуры в рассматриваемом наборе данных чаще всего сопровождается увеличением влажности.

Построим диаграмму рассеяния для визуального подтверждения результатов корреляционного анализа.

```
plt.figure(figsize=(8,6))

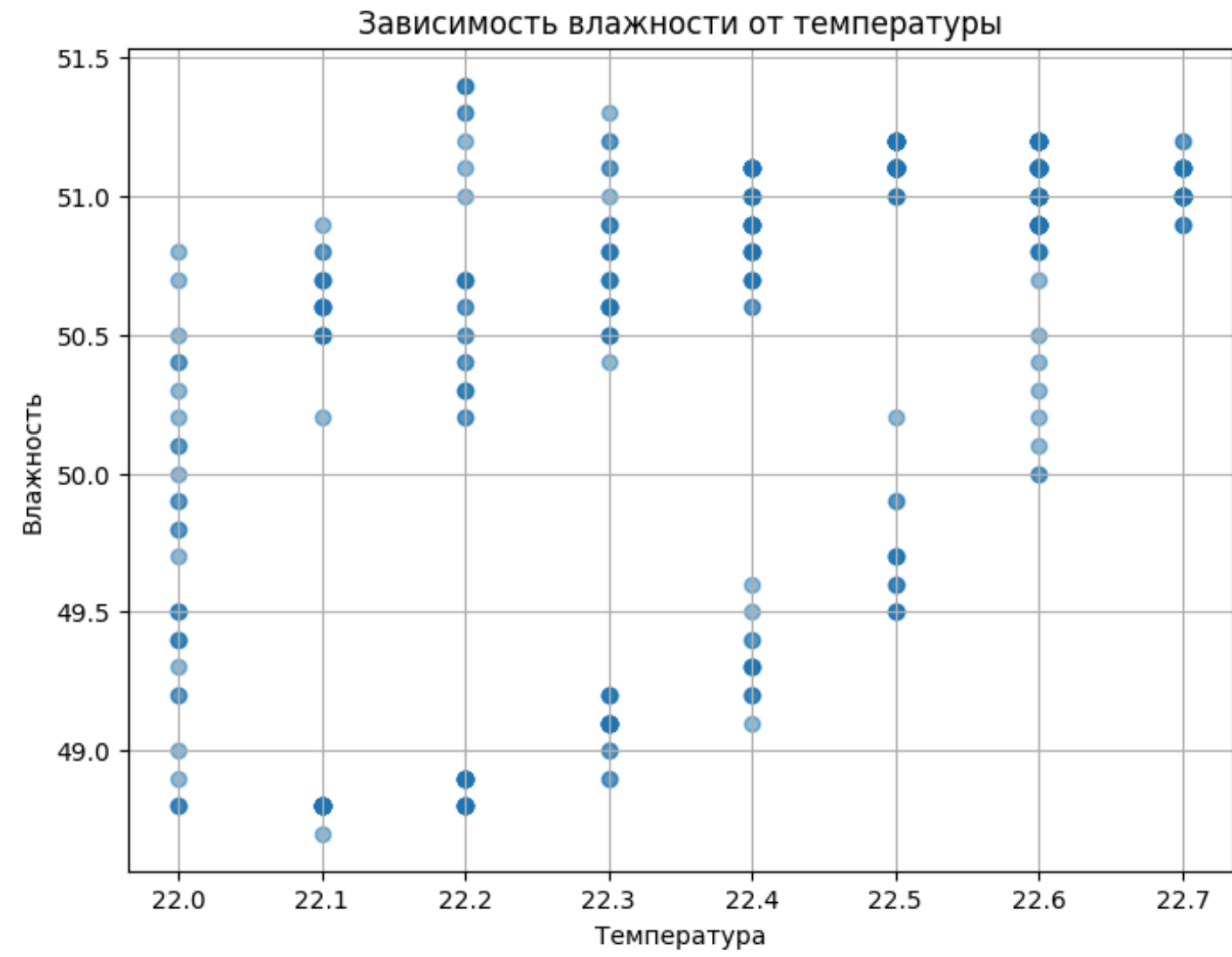
plt.scatter(
    sensor_data["temp"],
    sensor_data["humidity"],
    alpha=0.5
)

plt.title(
    "Зависимость влажности от температуры"
)

plt.xlabel("Температура")
plt.ylabel("Влажность")

plt.grid()

plt.show()
```



Точки на диаграмме образуют выраженную область концентрации с положительным направлением. Это подтверждает наличие статистической зависимости между температурой и влажностью, выявленной ранее с помощью коэффициента корреляции.

В ходе выполнения дополнительного задания были исследованы данные IoT-датчиков температуры и влажности воздуха.

В ходе выполнения дополнительного задания были исследованы данные IoT-датчиков температуры и влажности воздуха. Статистический анализ показал, что температура изменялась в узком диапазоне значений со средним значением 22,43 °C и стандартным отклонением 0,206 °C. Влажность характеризовалась большей изменчивостью: среднее значение составило 50,57 %, а стандартное отклонение 0,790 %. Для уменьшения влияния случайных колебаний были применены методы скользящего среднего и фильтра Калмана. Анализ результатов показал, что оба метода обеспечивают сглаживание сигналов, однако влияние фильтрации оказалось относительно небольшим. Это указывает на высокое качество исходных данных и отсутствие значительного уровня шумов. Корреляционный анализ выявил умеренно сильную положительную связь между температурой и влажностью. Коэффициент корреляции Пирсона составил 0,634. Построенная диаграмма рассеяния подтвердила наличие статистической зависимости между исследуемыми параметрами. Таким образом, проведенное исследование показало наличие взаимосвязи между параметрами окружающей среды и продемонстрировало возможности применения методов фильтрации для анализа данных реальных IoT-датчиков.

